Set-theoretic typing: structural, nominal, and the dial between

Status: **implemented (W3, D14).** This resolved the open shape of W3 (`docs/roadmap.typ`) and the surface question in Q3/D3 (`docs/decision-records.typ`); the encoding, surface, diagnostics, and tests landed in D14, and predicate sorts landed in D31. The note is kept as the argument, with the landed surface and the two deferrals called out inline.

Terminology. A *sort* in this document is a named value set; the code type is `EvaluatorValueSets.ValueSet`. It does not order anything.

The short version: Scale is already a set theory. What it does not yet have is the **label** that decides *which* equality a type uses — and that label is exactly what "structural vs nominal" means.

1. The starting point (before W3)

Four facts from the tree at the time of the design, stated so the rest follows from code and not taste.

A model is a finite set. `writeSet` (`components/prelude/evaluator/value-sets.fs`) stores a model as a tensor vector `[nameId, member₁, member₂, …]`; `SetsRoot` is an association vector of those. `model "small"` (`modelWord`) creates a set with no members; `admit` adds one.

Membership is structural. `containsMember` (`components/prelude/evaluator/value-sets.fs`) tests membership with `tensorA = tensorB` — F# structural equality over `{ Data; Shape }`. So `2 "small" is` is true because the *value* 2 is a member, not because a particular reference is. Two tensors with the same contents are the same member.

The algebra is set algebra, and it is property-pinned. `nor` is `U \ (A ∪ B)`; `and`/`or` are ∩/∪. `properties.fs` pins this: *"`nor` is set complement over an explicit universe"* and *"`and`/`or` agree with intersection and union"*, over random universes. This is the Tier-1 commitment in `docs/commitments-ledger.typ`.

Before W3 there was no sort. A model had no declared element type. That is *why* `nor` took a universe operand: there was nothing to default the universe to. The ternary `nor` (D3) was the minimal correct fix, and D3 says so — *"The universe is passed explicitly … at which point the universe could default to a sort."* W3 now supplies that default as the unary `complement` over the sort's declared domain, while leaving `nor` ternary.

There is also a second, quieter fact worth naming:

The substrate is already half nominal. `EntryIdentity` is a distinct address (host view), and `addText` **interns** — equal text yields the *same* identity — while `addTensor` does not. So identity is nominal for text and fresh for tensors. The language already lives at the structural/nominal boundary; it just does not say so.

One more: `ScaleFile.Guard` is a predicate — `GreaterThan`, `AtLeast`, `IsTrue`, … — and `Refiner.evaluateGuard` evaluates it. A guard is a set comprehension: it *narrows* the admissible inputs. The refiner's whole job is to discover the guard that closes a class. So refinement already exists, in the refinement loop, without being called a type.

2. The two endpoints, and why they are one object

A **sort** is a set that may also carry a name. Write it as a pair:

`ValueSet ≜ ( N : option Name , X ⊆ U )`

`N` is the nominal component; `X`, the extension, is the structural component.

- **Nominal equality** compares by `N`: `(N₁, _) ≡ₙₒₘ (N₂, _) ⇔ N₁ = N₂`. Two sorts are the same sort iff they have the same name, whatever their members.

- **Structural equality** compares by `X`: `(_, X₁) ≡ₛₜᵣ (_, X₂)  ⇔  X₁ = X₂`. Two sorts are the same sort iff they have the same extension, whatever their names. (That is *extensionality*, and it is the axiom of set theory, not an analogy for it.)

Both are facts about the **same pair**. Nominal and structural are not two type systems with a translator between them; they are two **projections of one record**. That is the whole reason set theory can host both without seams.

3. The dial, and "and more"

The two projections are the endpoints of one dial, and the set operations are the teeth that move along it.

Narrow toward nominal — add or rely on `N`:

- seal a set with a name (`narrow`): equality becomes name-based, so two structurally identical sets are *distinct* types. This is `newtype`, an opaque handle, a unit of measure.
- restrict the extension: fewer members, a smaller `X`.

Widen toward structural — drop `N` and compare by `X`:

- erase the name (`widen`): equality becomes extensional, so two names for the same set are the *same* type. This is `type`-alias behaviour, duck typing.
- enlarge the extension: more members, a larger `X`.

And, beyond the two endpoints, the same value yields more — every one of these is already a set-theoretic operation, so none of them is a bolt-on:

Notion	Set-theoretic form	Already present?
Subtyping	`A <: B  ⇔  A ⊆ B`	no; `⊆` is not a word
Intersection types	`A ∩ B` (`and`)	yes — `andWord`
Union types	`A ∪ B` (`or`)	yes — `orWord`
Complement type / top	`¬(A∪B)` over `U` (`nor`); `U` is top	yes — `norWord`
Bottom	`∅`	yes — `model` with no `admit`
Refinement types	`{ x ∈ U | P x }`	yes — `Guard` + `evaluateGuard`
Co-product / sum	tagged disjoint union `A ⊎ B`	no
Multisets	membership with **multiplicity**	no — `isMemberId` dedups
Sequences / order	a set with a position function	no — but `Tensor` is ordered
Recursive types	the least fixed point of `X ↦ F(X)`	no

Read the bottom rows as the honest boundary of the claim. A set has *no multiplicity and no order*; widening past sets is exactly where you add them (the `Tensor`'s ordered `Data` is already the raw material for both). So: **set theory narrows to nominal, widens to structural, and widens further to multiset and sequence** — which is to say it is not a compromise between the two type systems but the ground both stand on.

One consequence worth stating plainly, because it corrects a common framing: "structural vs nominal" is not a choice between `Set` and `Map`. It is the choice of **which projection a comparison reads**, and it can be made *per comparison site* rather than once for the language. Set theory lets the language hold both readings of a single value.

4. Syntax

The user's stated preference is **special syntax** that makes the structural/nominal reading visible at the point of use. The design below honours that, and it fits the existing Forth-flavoured stack surface (`1 "small" admit`, `U A B nor` — operands then word).

Two sigils carry the whole dial, and both were free before W3. `?x` (logic variable), `"…"` (text), and `{ … }` (quotation) are taken; `[ ]` and a leading `:` were not, so W3 took them. The tokenizer splits both out of a word (`Tokenizer.splitSigils`).

Sigil	Reads as	Reading
`[ … ]`	a **structural** set — an extension, anonymous	"the set of …"
`:Name`	a **nominal** seal — a name on a set	"the thing called Name"

4.1 The structural base

```
[ 1 2 3 ]               \ extension: {1,2,3}
[ 1 2 add ]             \ the body is evaluated: {3}
[ ]                     \ the empty set  (∅)

A B subset              \ A ⊆ B          ( -- bool )
A B union               \ A ∪ B
A B intersect           \ A ∩ B
A B except              \ A \ B
U A B nor               \ U \ (A ∪ B)    (existing, unchanged)
x S member              \ x ∈ S
A B same-shape          \ extensional equality  ( ≡str )
```

These are **structural throughout**: they compare by extension. Nothing here needs a sort. Comprehension syntax (`[ ?x : ?x > 1 ]`) is **not landed**; the literal evaluates its body and captures the resulting values, so a filter needs a word that does not exist yet.

4.2 The nominal layer

Operands come first, as everywhere else in the stack language. The design note originally wrote `:player [ ]`; the landed form is `[ ] :player seal`, because a prefix name would need a pending-seal side channel — host state the level model forbids. `:player` remains a *reference* to an existing sort.

```
[ ] :player seal        \ a sort named player, empty extension
[ 1 2 3 ] :player seal  \ a sort named player over {1,2,3}
:player                 \ the sort named player, referenced

S unname                \ widen a sort back to a bare set (drop the name)
A B same-name           \ nominal equality  ( ≡nom )
A B sort-of             \ is B the sort of A?  (nominal membership of a sort)
A B merge               \ sort-preserving union; ValueSetMismatch on different seals
S complement            \ declared sort domain \ S; ValueSetNotClosed if anonymous
```

`S :Name seal` is therefore the same operation written over an arbitrary set `S`; it is the general form of the literal examples above. Now the reading is local: `same-shape` asks about members, `same-name` asks about the label, and the two are visibly different words over the same pair.

4.3 Why this settles Q3

Q3 asked whether a model's surface should be `model <sort> <name>`, a separate `sort` word, or name-as-sort. The set-theoretic view answers it:

A sort is `(Name, Extension)`. A surface must be able to express **both components**, so it needs a word for the set and a distinct word for the label. **Name-as-sort** (`model <name>`, today) collapses the pair to the name alone, which is precisely the structural half thrown away — and with it any cross-sort comparison, and therefore all of W10 (multiple dispatch).

So the landed decision is the **separate `seal` word** (with `[ … ]` for the set), and the reason is not aesthetic: the pair has two components, and W10 (landed as D19) dispatches on the **extension** while W4 unifies under the **name**.

4.4 Migration, keystroke by keystroke

Nothing above is a rewrite of the model layer; it is a *naming* of what the model layer already computes.

Today	Becomes	Note
`"small" model`	`[ ] :small seal`	`model` is sugar for "empty declared sort plus a model under that name"; both remain
`1 "small" admit`	`1 :small admit`	unchanged algorithm; the name is a sort reference
`2 "small" is`	`2 :small is`	structural membership, same `isMemberId`; `member` is the set-first spelling
`U A B nor`	unchanged	already set algebra
—	`[ ] :player seal`	the whole of W3's new surface

The two costs, both paid:

1. **The tokenizer knows the sigils.** `[`/`]`/`:Name` are split out alongside `?x` and `"…"` (`Tokenizer.splitSigils`), so `[1 2]:s` tokenizes without requiring spaces around the brackets.
2. **`seal` changes equality for the sealed value**, so it is a *new* entry on the stack, never a mutation of the set beneath — the same immutability discipline the image already enforces.

4.5 What this is not

- **Not child-facing.** `[ ]`, `:Name`, `nor`, `seal` are the *lab* surface. The child-facing product remains the sentence layer (`Every player that is alive can jump.`), which compiles down to these. Confirmed in this project's own answers: the sentence layer is the product; the stack is the bench.
- **Not a departure from the level model.** A sort is `(option EntryIdentity, EntryIdentity list)` — both components project into tensors, so the host type holds no state absent from the image. `ValueTag`'s nominal component — the sort name — is now image-resident (W3), so no separate host `ValueTag` is needed.

4.6 A decision table is a partition by comprehensions

The optional decision-table layer (D13, `docs/decision-models.typ`) is a direct consequence of this reading, which is why it is recorded here rather than only in its own note.

A row's guards are comprehensions over the input sort. `GreaterThan("health", 2)` is `{ subject | subject.health > 2 }`, a refinement sort. A decision table is then a **finite family of comprehensions** $g_1 \dots g_n$ over one sort `S`, and its two proved properties are set statements:

Table property	Set statement	Z3 query
completeness	$g_1 \cup \dots \cup g_n = S$	`satisfiable` $(\neg(g_1 \vee \dots \vee g_n))$ is `unsat`
disjointness (`Unique`)	$g_i \cap g_j = \varnothing$ for `i < j`	`satisfiable` $(g_i \wedge g_j)$ is `unsat`

So a "gap" is a member of `S` outside the union, and an "overlap" is a member of a pairwise intersection. `DecisionTable.check` is the sort-level statement of facts the dial already names; it does not add a new concept, it applies the existing one to an ordered family.

Two honest qualifications. First, `Inputs` still carries `DomainExpression.FieldType`, not the D12 `(option Name, Extension)` record: W3 landed the sort registry, but the optional decision-table layer is not wired to it (D13 remains unwired by design). The correspondence is conceptual, and would become literal only if the table layer is adopted. Second, the table is *optional*: no sort, parser, or emitter depends on it, and a domain that never declares one is unaffected.

4.7 Unification reads the name (W4/D15)

A typed term is the pair `(N, t)`: an inner term and a nominal seal. The W4 unifier compares `N` directly and then unifies `t`; it never asks whether the two extensions happen to overlap, so a value cannot drift from one named sort to another merely because the sets intersect. That is the point of making the name a real component instead of a label on a set.

```
"player" model
?x :player typed                \ (player, ?x)
?x :player typed 5 :player typed unify   \ 1
?x :player typed ?y :enemy typed unify   \ 0 — names differ
?x :player typed ?y unify                \ ?y inherits (player, ?x)
?x :player typed 5 unify                 \ 0 — typed vs unsorted non-variable
```

A bare variable is deliberately allowed to bind to a typed term: it is the "unknown at this sort" case, and the binding gives it the sort. A typed term is not allowed to bind to an untyped non-variable, because that would silently erase the seal. The extension remains meaningful, but its consumer is W10 dispatch, not W4 equality.

The term encoding keeps the substitution image-resident: variables are text entries, compounds are `-2` tagged vectors, typed terms are `-3` tagged vectors, and `BindingsRoot` remains the association vector. No host type holds a substitution that the image cannot reconstruct.

5. Where it lands

- **W3 — landed (D14).** The sort registry is the pair `(option Name, Extension)`; `N` is an image entry, `X` is the member list; `ValueSetMismatch` and `ValueSetNotClosed` are named diagnostics; the default universe is the unary `complement` over a declared sort. The ternary `nor` is unchanged: a binary `nor` cannot be distinguished on a stack, so D3's open line is closed by a distinct word rather than an arity change.
- **W4 — landed (D15).** `unify` is first-order and structural: `walk`, occurs-check, bidirectional bindings, compound arguments, recursive substitution through `resolve`/`lookup`, and nominal equality for `typed` terms. The name `N` is what W4 reads; the extension `X` is untouched.
- **W10 — landed (D19).** Dispatch reads `X` (the extension), not name-as-sort: a typed operand uses its seal, an untyped operand infers the unique most-specific declared sort containing it, and specificity is extension inclusion.
- **Predicate sorts — landed (D31).** A sort may also carry an image-resident membership quotation: `base { predicate } :Name predicate-seal`. The extension is the base; the quotation narrows it, and membership is a three-way verdict (`Member`, `NotMember`, `Undecided reason`) rather than a silent false. `is`, `member`, and `subset` read the verdict; operations that would need the whole narrowed extension (`union`, `intersect`, `except`, `same-shape`, `merge`, `complement`, `and`/`or`/`nor`, and `admit`) refuse with `ValueSetUndecided`. This is the first sort *constructor* that is data: a user adds a refinement sort without editing F#.

- **`Guard`** is retired as a special case and recognised for what it is: a comprehension, that is, a refinement sort. D31 lands the predicate sort as the data-driven refinement, and D34 lands `sort { predicate } filter` as the filtering word; the `[ … ]` literal still evaluates its body, and the `[ ? x : … ]` surface needs a domain decision.

6. The commitment, in one sentence

> Scale's types are sets; a sort adds a name to a set; nominal equality reads the name, structural equality reads the members, and the set operations — ∩, ∪, , ⊆, ⊇, refinement, and the multiplicity extension — move between them.

That is checkable, and it leaves the property suite's job unchanged: `nor` is still set complement, `and`/`or` still agree with ∩/∪.